



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий

Кафедра КБ-4 «Интеллектуальные системы информационной безопасности»

Дисциплина «Технологии извлечения знаний из больших данных»

**Отчет
о проделанной практической работе**

Выполнил студент 1 курса
Группы: ББМО-01-25
*Мухаметшин Александр
Ринатович*

Москва
2025

ОГЛАВЛЕНИЕ

ТЕОРЕТИЧЕСКОЕ ВВЕДЕНИЕ	3
ЗАДАНИЕ	9
ХОД РАБОТЫ	11
ВЫВОД.....	24
ПРИЛОЖЕНИЕ А.....	25

ТЕОРЕТИЧЕСКОЕ ВВЕДЕНИЕ

При разработке SVM-классификатора используется учебный набор данных: $U = \{(z_1, y_1), \dots, (z_s, y_s)\}$, в котором каждый кортеж (z_i, y_i) содержит информацию об объекте $z_i \in Z$ и число $y_i \in Y = \{-1, +1\}$, определяющее метку класса, к которому принадлежит объект z_i . Набор объектов Z представляет собой объединение набора объектов Z^- , метка класса которых принимает значение «-1», и набора объектов Z^+ , метка класса которых принимает значение «+1», т.е. $Z = Z^+ \cup Z^-$. Каждый объект $z_i \in Z$ представлен q -мерным вектором числовых характеристик $z_i = (z_{i1}, \dots, z_{iq})$, (нормированных значениями из отрезка $[0, 1]$), где z_{il} – числовое значение l -й характеристики для i -го объекта ($i = 1, \dots, s, l = 1, \dots, q$).

Вышеуказанный учебный набор данных U многократно случайным образом разбивается на обучающую и тестовую выборки, состоящие соответственно из S и $s - S$ кортежей ($s > S$), с целью реализации многократного обучения и тестирования формируемых в результате SVM-классификаторов с последующим определением лучшего классификатора в смысле обеспечения максимально возможной точности классификации. При этом для каждого SVM-классификатора определяются тип функции ядра $\kappa(z_i, z_t)$, значения параметров функции ядра и значение параметра регуляризации $C > 0$, позволяющего найти компромисс между максимизацией ширины полосы, разделяющей классы, и минимизацией суммарной ошибки.

При разработке SVM-классификатора с применением функции ядра $\kappa(z_i, z_t)$ определяется классифицирующая функция $F: Z \rightarrow Y: y_i = F(z_i)$, устанавливающая для объекта $z_i \in Z$ его класс принадлежности $y_i \in Y = \{-1; +1\}$. В качестве функции ядра $\kappa(z_i, z_t)$ обычно используются следующие функции:

- линейная $\kappa(z_i, z_t) = z_i \cdot z_t$;
- полиномиальная $\kappa(z_i, z_t) = (1 + z_i \cdot z_t)^d$;
- радиальная базисная $\kappa(z_i, z_t) = \exp(- \|z_i - z_t\|^2 / (2\sigma^2))$;

– сигмоидная $\kappa(z_i, z_\tau) = \tanh(k_2 z_i \cdot z_\tau + k_1)$,

где $z_i \cdot z_\tau$ – скалярное произведение векторов z_i и z_τ ; $d \in \mathbb{N}$ (по умолчанию $d = 3$), $\sigma > 0$ (по умолчанию $\sigma = 1/2$), $k_2 < 0$ (по умолчанию $k_2 = -1/2$) и $k_1 > 0$ (по умолчанию $k_1 = 1$) – некоторые параметры; $\tanh$ – гиперболический тангенс.

В случае линейной разделимости классов в результате обучения SVM-классификатора строится гиперплоскость, разделяющая объекты из Z на два класса: $w \cdot z + b = 0$, где w – вектор нормали к гиперплоскости; b – параметр, задающий смещение гиперплоскости относительно начала координат; $w \cdot z$ – скалярное произведение вектора w нормали к гиперплоскости и вектора характеристик некоторого объекта z . Условие $|w \cdot z + b| < 1$ задает полосу, которая разделяет классы. Чем шире эта полоса, тем увереннее можно классифицировать объекты. Для максимизации ширины полосы $2 / \|w\|$ таким образом, чтобы внутри ее не попал ни один объект из обучающей выборки, должна быть решена задача квадратичной оптимизации: $\min \|w\|^2$ s.t. $y_i (w \cdot z_i + b) \geq 1, i = 1, \dots, S$. (1)

В случае линейной неразделимости классов задача построения разделяющей гиперплоскости (с учетом теоремы Куна – Таккера) сводится к задаче квадратичного программирования, содержащей только двойственные переменные λ_i ($i = 1, \dots, S$): $\min_{\lambda} L(\lambda) = 1/2 \sum_{i=1}^S \sum_{j=1}^S y_i y_j \lambda_i \lambda_j \kappa(z_i, z_j) - \sum_{i=1}^S \lambda_i$ s.t. $0 \leq \lambda_i \leq C, i = 1, \dots, S, \sum_{i=1}^S y_i \lambda_i = 0$. (2)

В результате обучения SVM-классификатора определяются опорные векторы, являющиеся векторами характеристик тех объектов z_i из обучающей выборки, для которых значения соответствующих им двойственных переменных $\lambda_i \neq 0$. Опорные векторы находятся ближе всего к разделяющей гиперплоскости и несут всю информацию о разделении классов.

В результате обучения определяется правило классификации, а именно классифицирующая функция, устанавливающая для произвольного объекта z его класс принадлежности с меткой «-1» или «+1»: $F(z) = \text{sign}(\sum_{i=1}^S \lambda_i y_i$

$\kappa(z_i, z) + b$), (3) где $b = y_k - w \cdot z_k; \sum_{i=1}^S \lambda_i y_i z_i = w$.

При этом суммирование в правиле (3) выполняется только по опорным векторам (то есть по векторам, для которых $\lambda_i \neq 0$).

Основная проблема, возникающая при разработке SVM-классификатора, связана с отсутствием рекомендаций по выбору типа функции ядра $\kappa(z_i, z)$, значений параметров функции ядра и значения параметра регуляризации C , при которых будет обеспечена высокая точность классификации объектов. Данная проблема может быть решена с применением тех или иных оптимизационных алгоритмов, например, с использованием PSO-алгоритма, хорошо зарекомендовавшего себя при решении широкого спектра задач оптимизации, а также с использованием метода перебора искомых значений параметров по сетке.

Оценка качества классификации может быть выполнена с применением различных показателей качества классификации, среди которых следует выделить: показатель общей точности (Accuracy, Acc), называемый также показателем общего успеха (Overall Success Rate, OSR); показатель чувствительности (Sensitivity, Se), называемый также показателем полноты (Recall, Re); показатель специфичности (Specificity, Sp); показатель точности (Precision, Pr); а также показатель сбалансированной F-меры (F-measure, F1), которые вычисляются в соответствии с формулами: $OSR = (TP + TN) / (TP + FP + TN + FN)$, (4) $Se = TP / (TP + FN)$, (5) $Sp = TN / (TN + FP)$, (6) $Pr = TP / (TP + FP)$, (7) $F1 = 2 \cdot Pr \cdot Re / (Pr + Re)$, (8) где TP – число истинно положительных наблюдений; TN – число истинно отрицательных наблюдений; FP – число ложноположительных наблюдений («ложных обнаружений», ошибка II рода); FN – число ложноотрицательных наблюдений («ложных пропусков», ошибка I рода); $Re = Se$.

Показатель общей точности OSR показывает долю истинно предсказанных наблюдений по отношению к общему числу наблюдений классификатора.

Показатель чувствительности Se показывает, какая доля от общего

числа реальных положительных наблюдений предсказана в качестве положительных, то есть говорит о том, насколько классификатор «пессимистичен» в своих оценках или как часто он «отбрасывает» (а это происходит при низком значении показателя Se) наблюдения нужного класса. Этот показатель также называют показателем полноты Re .

Показатель специфичности Sp показывает, какая доля от общего числа реальных отрицательных наблюдений предсказана в качестве отрицательных.

Показатель точности Pr показывает, сколько из предсказанных положительных наблюдений являются действительно положительными, т.е. говорит о том, насколько классификатор оптимистичен в своих оценках или как часто он «предпочитает» (а это происходит при низком значении показателя Pr) присоединять наблюдения других классов к заданному.

Показатель сбалансированной F -меры ($F1$) вычисляет гармоническое среднее между показателем точности Pr и показателем полноты Re . При этом в формуле (8) этим показателям приписан одинаковый вес.

Особенности разработки SVM-классификатора: – реализуется только бинарная классификация объектов (то есть возможно разбиение объектов только на 2 класса); – в явном виде формируется правило классификации (3), устанавливающее для произвольного объекта z его класс принадлежности с меткой «-1» или «+1».

Хотя базовый SVM-алгоритм реализует только бинарную классификацию, он может быть применен для выполнения мультиклассовой классификации при использовании стратегий выбора решений «один против всех» (one-against-all) и «каждый против каждого» (one-against-one). В первом случае строится R классификаторов так, что каждый класс r ($r = 1, \dots, R$) сопоставляется с остальными ($R - 1$) классами, то есть в каждом из R случаев выбор осуществляется из двух вариантов: «класс r » и «не класс r ». Итоговое решение по всем классам принимается по схеме «победитель забирает всё» (winner takes all). Во втором случае классификаторы строятся для каждой пары классов для того, чтобы можно было однозначно разделить

любые два класса. Количество классификаторов при этом равно $R(R-1)/2$. Каждый из обученных классификаторов определяет для классифицируемого объекта информацию о его принадлежности одному из двух классов, участвовавших в обучении. К полученному множеству решений о классовой принадлежности объекта применяется правило большинства голосов, и класс, выбранный большинством классификаторов, принимается за искомый.

Для получения оценок значений параметров обучения и для оценки обобщающей способности алгоритмов, обучаемых по прецедентам, можно использовать кросс-проверку (скользящий контроль, Cross-Validation).

Суть кросс-проверки заключается в следующем. Пусть имеется правило классификации $A: Z \rightarrow Y$. Учебный набор объектов Z , использующийся для построения классификатора, разбивается на V приблизительно равных по объему непересекающихся поднаборов Z_v^k (Z_v^k – v -й поднабор длины k_v , $\sum_{v=1}^V k_v = s$, v – номер разбиения, $v = 1, \dots, V$): $Z_v^k \cap Z_t^k = \emptyset$ ($v \neq t$), каждый из которых поочередно будет представлять тестовую выборку, а остальные $V - 1$ поднаборов составят обучающую выборку. Для каждого v -го разбиения выполняется настройка значений параметров алгоритма по обучающей выборке, затем оценивается его средняя ошибка на объектах тестовой выборки. Оценкой скользящего контроля называется средняя по всем разбиениям величина ошибки на тестовых выборках.

Экспериментальные исследования, проведенные на различных множествах данных для различных алгоритмов классификации, показали, что при $V = 10$ (10-кратной перекрестной проверки) достигается минимальное значение ошибки. Если выборка независима, то средняя ошибка скользящего контроля дает несмещенную оценку вероятности ошибки. Это выгодно отличает ее от средней ошибки на обучающей выборке, которая может оказаться смещенной (оптимистически заниженной) оценкой вероятности ошибки, что связано с явлением переобучения. Скользящий контроль является стандартной методикой тестирования и сравнения методов

классификации, регрессии и прогнозирования.

В качестве лучшего разбиения на поднаборы принимается разбиение, обеспечивающее минимальную ошибку на тестовой выборке.

ЗАДАНИЕ

Разработать SVM-классификатор (метод опорных векторов) для набора данных. В качестве языка программирования использовать Python.

Разбить выборку на обучающую и тестовую.

Обучить, проверить качество классификатора на обучающей и тестовой выборках: рассчитать общую точность, Recall, Precision, F1-measure.

Оценить число опорных векторов.

Рассмотреть различные типы ядра (линейное, RBF, полиномиальное, сигмоидное (тангенсальное)), различные сочетания значений параметра регуляризации C и параметров ядра.

Перебор по сетке (grid search).

На основе реальных данных, взятых из репозитория задач машинного обучения – UCI Machine Learning Repository и традиционно используемых для тестирования алгоритмов машинного обучения, выполните следующие действия:

- 1) найдите по указанной ссылке в UCI-репозитории набор данных в соответствии с предложенным в таблице вариантом;
- 2) импортируйте набор данных в рабочий лист пакета Statistica;
- 3) определите значения параметров входных переменных: укажите тип каждой переменной, шкалу измерения, формат отображения;
- 4) запустите процедуру построения SVM-классификатора;
- 5) укажите тип переменных для анализа (независимые переменные и категориально-зависимую переменную);
- 6) разбейте исходный набор данных случайным образом на обучающую и тестовую выборки [размер (число наблюдений) тестовой выборки должен составлять от 1/10 до 1/3 от размера исходного набора данных];
- 7) на вкладке SVM установите для проведения анализа тип модели, используемый для построения SVM-классификатора;
- 8) определите тип используемой при обучении функции ядра и

значения ее параметров;

9) включите в процесс обучения процедуру перекрестной проверки и установите значение кратности перекрестной проверки;

Повторите пп.6 – 10 для построения нескольких моделей (используйте различное соотношение числа объектов в обучающей и тестовой выборках, различные типы SVM-классификаторов, различные типы функций ядра и значений их параметров), сравните качество обученных классификаторов.

По результатам обучения трех лучших SVM-классификаторов подготовьте отчет, в который включите информацию о результатах моделирования, рассмотренную в практической части. По каждой модели представьте 4 – 6 графических обзоров результатов моделирования.

В качестве датасета был взят вариант 8 – <http://archive.ics.uci.edu/ml/datasets/Heart+Disease>.

ХОД РАБОТЫ

Часть 1. Реализация кода

На рисунке 1 импортируются необходимые библиотеки для работы с массивами, графиками, таблицами и комбинациями.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
from ucimlrepo import fetch_ucirepo
```

Рисунок 1 – Импорт необходимых библиотек

На рисунке 2 осуществляется загрузка необходимого датасета.

```
heart_disease = fetch_ucirepo(id=45)
X = heart_disease.data.features
y = heart_disease.data.targets

print(f"\nРазмер датасета: {X.shape[0]} наблюдений, {X.shape[1]} признаков")
print(f"\nПервые 5 строк данных:")
print(X.head())
print(f"\nЦелевая переменная:")
print(y.head())
```

Рисунок 2 – Загрузка датасета Heart Disease

Далее определяем параметры переменных.

Описание параметров приведено ниже:

ОПРЕДЕЛЕНИЕ ПАРАМЕТРОВ ПЕРЕМЕННЫХ

1. age (возраст): Числовая, интервальная шкала, годы
2. sex (пол): Категориальная, номинальная, 0=женщина, 1=мужчина
3. cp (тип боли в груди): Категориальная, порядковая, 1-4
4. trestbps (давление покоя): Числовая, интервальная, мм рт.ст.
5. chol (холестерин): Числовая, интервальная, мг/дл
6. fbs (сахар крови): Категориальная, номинальная, 0/1
7. restecg (ЭКГ покоя): Категориальная, порядковая, 0-2
8. thalach (макс. пульс): Числовая, интервальная
9. exang (стенокардия): Категориальная, номинальная, 0/1
10. oldpeak (депрессия ST): Числовая, интервальная
11. slope (наклон ST): Категориальная, порядковая, 1-3
12. ca (сосуды): Числовая, дискретная, 0-3
13. thal: Категориальная, порядковая, 3/6/7

ЦЕЛЕВАЯ ПЕРЕМЕННАЯ (num):

- 0: отсутствие заболевания (<50% сужение)
- 1-4: наличие заболевания (>50% сужение)

На рисунке 3 показан код определения параметров.

```

print("\nИнформация о типах данных:")
print(X.dtypes)

print("\nСтатистическое описание данных:")
print(X.describe())

print(f"\nПропущенные значения:\n{X.isnull().sum()}")
X = X.fillna(X.median())

y_binary = (y > 0).astype(int)
print(f"\nРаспределение целевой переменной:")
print(y_binary['num'].value_counts())

```

Рисунок 3 – определение параметров

На рисунке 4 показан код подготовки данных для SVM.

```

print("\nНезависимые переменные (13 признаков):", list(X.columns))
print("Зависимая переменная: num (наличие сердечного заболевания)")
|
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

```

Рисунок 4 – подготовка и нормализация данных

Далее была подготовлена и протестирована модель Linear Kernel с разбиением 70/30. Код показан на рисунке 5.

```

print("МОДЕЛЬ 1: Linear Kernel, разбиение 70/30")

X_train1, X_test1, y_train1, y_test1 = train_test_split(
    X_scaled, y_binary, test_size=0.3, random_state=42
)
print(f"Обучающая выборка: {len(X_train1)} наблюдений")
print(f"Тестовая выборка: {len(X_test1)} наблюдений")

svm_linear = SVC(kernel='linear', C=1.0, random_state=42)
svm_linear.fit(X_train1, y_train1.values.ravel())

y_pred1 = svm_linear.predict(X_test1)

acc1 = accuracy_score(y_test1, y_pred1)
cv_scores1 = cross_val_score(svm_linear, X_train1, y_train1.values.ravel(), cv=5)

print(f"\nТочность на тестовой выборке: {acc1:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores1.mean():.4f} (+/- {cv_scores1.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test1, y_pred1))

results.append({
    'model': 'Linear (70/30)',
    'kernel': 'linear',
    'C': 1.0,
    'split': '70/30',
    'test_acc': acc1,
    'cv_mean': cv_scores1.mean(),
    'cv_std': cv_scores1.std()
})

```

Рисунок 5 – модель Linear Kernel, разбиение 70/30

Далее была подготовлена и протестирована модель RBF SVM (RBF Kernel) с параметрами: разбиение 80/20, C=10, gamma=0.1. Код показан на рисунке 6.

```

print("МОДЕЛЬ 2: RBF Kernel, разбиение 80/20, C=10, gamma=0.1")

X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X_scaled, y_binary, test_size=0.2, random_state=42
)
print(f"Обучающая выборка: {len(X_train2)} наблюдений")
print(f"Тестовая выборка: {len(X_test2)} наблюдений")

svm_rbf = SVC(kernel='rbf', C=10.0, gamma=0.1, random_state=42)
svm_rbf.fit(X_train2, y_train2.values.ravel())

y_pred2 = svm_rbf.predict(X_test2)
acc2 = accuracy_score(y_test2, y_pred2)
cv_scores2 = cross_val_score(svm_rbf, X_train2, y_train2.values.ravel(), cv=5)

print(f"\nТочность на тестовой выборке: {acc2:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores2.mean():.4f} (+/- {cv_scores2.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test2, y_pred2))

results.append({
    'model': 'RBF (80/20, C=10)',
    'kernel': 'rbf',
    'C': 10.0,
    'split': '80/20',
    'test_acc': acc2,
    'cv_mean': cv_scores2.mean(),
    'cv_std': cv_scores2.std()
})

```

Рисунок 6 – модель RBF Kernel, разбиение 80/20, C=10, gamma=0.1

Далее была подготовлена и протестирована модель RBF SVM (RBF Kernel) с параметрами: разбиение 75/25, C=1, gamma='scale'. Код показан на рисунке 7.

```

print("МОДЕЛЬ 3: RBF Kernel, разбиение 75/25, C=1, gamma='scale'")

X_train3, X_test3, y_train3, y_test3 = train_test_split(
    X_scaled, y_binary, test_size=0.25, random_state=42
)
print(f"Обучающая выборка: {len(X_train3)} наблюдений")
print(f"Тестовая выборка: {len(X_test3)} наблюдений")

svm_rbf2 = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm_rbf2.fit(X_train3, y_train3.values.ravel())

y_pred3 = svm_rbf2.predict(X_test3)
acc3 = accuracy_score(y_test3, y_pred3)
cv_scores3 = cross_val_score(svm_rbf2, X_train3, y_train3.values.ravel(), cv=5)

print(f"\nТочность на тестовой выборке: {acc3:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores3.mean():.4f} (+/- {cv_scores3.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test3, y_pred3))

results.append({
    'model': 'RBF (75/25, C=1)',
    'kernel': 'rbf',
    'C': 1.0,
    'split': '75/25',
    'test_acc': acc3,
    'cv_mean': cv_scores3.mean(),
    'cv_std': cv_scores3.std()
})

```

Рисунок 7 – RBF Kernel, разбиение 75/25, C=1, gamma='scale'

Далее была подготовлена и протестирована модель Polynomial SVM с параметрами: разбиение 70/30, degree=3. Код показан на рисунке 8.

```

print("МОДЕЛЬ 4: Polynomial Kernel, разбиение 70/30, degree=3")

svm_poly = SVC(kernel='poly', degree=3, C=1.0, random_state=42)
svm_poly.fit(X_train1, y_train1.values.ravel())

y_pred4 = svm_poly.predict(X_test1)
acc4 = accuracy_score(y_test1, y_pred4)
cv_scores4 = cross_val_score(svm_poly, X_train1, y_train1.values.ravel(), cv=5)

print(f"\nТочность на тестовой выборке: {acc4:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores4.mean():.4f} (+/- {cv_scores4.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test1, y_pred4))

results.append({
    'model': 'Poly (70/30, deg=3)',
    'kernel': 'poly',
    'C': 1.0,
    'split': '70/30',
    'test_acc': acc4,
    'cv_mean': cv_scores4.mean(),
    'cv_std': cv_scores4.std()
})

```

Рисунок 8 – Polynomial Kernel, разбиение 70/30, degree=3

Далее была подготовлена и протестирована модель Sigmoid SVM с разбиением 75/25. Код показан на рисунке 9.

```
print("МОДЕЛЬ 5: Sigmoid Kernel, разбиение 75/25")

svm_sigmoid = SVC(kernel='sigmoid', C=1.0, random_state=42)
svm_sigmoid.fit(X_train3, y_train3.values.ravel())

y_pred5 = svm_sigmoid.predict(X_test3)
acc5 = accuracy_score(y_test3, y_pred5)
cv_scores5 = cross_val_score(svm_sigmoid, X_train3, y_train3.values.ravel(), cv=5)

print(f"\nТочность на тестовой выборке: {acc5:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores5.mean():.4f} (+/- {cv_scores5.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test3, y_pred5))

results.append({
    'model': 'Sigmoid (75/25)',
    'kernel': 'sigmoid',
    'C': 1.0,
    'split': '75/25',
    'test_acc': acc5,
    'cv_mean': cv_scores5.mean(),
    'cv_std': cv_scores5.std()
})
```

Рисунок 9 – Sigmoid Kernel, разбиение 75/25

Далее был написан код вывода сравнительной таблицы и поиска трех наилучших моделей. Данный код показан на рисунке 10.

```
print("СРАВНИТЕЛЬНАЯ ТАБЛИЦА МОДЕЛЕЙ")

results_df = pd.DataFrame(results)
results_df = results_df.sort_values('test_acc', ascending=False)
print("\n", results_df.to_string(index=False))

print("ТРИ ЛУЧШИЕ МОДЕЛИ:")
top3 = results_df.head(3)
for idx, row in top3.iterrows():
    print(f"\n{row['model']}:")
    print(f" - Kernel: {row['kernel']}")
    print(f" - Test Accuracy: {row['test_acc']:.4f}")
    print(f" - CV Score: {row['cv_mean']:.4f} ± {row['cv_std']:.4f}")
```

Рисунок 10 – сравнение моделей

Далее были визуализированы графики – сравнение точности (рисунок 11), распределение целевой переменной (рисунок 12), Confusion Matrix лучших моделей (рисунок 13) и сравнение метрик precision/recall/f1 у лучших моделей (рисунок 14).


```

fig, axes = plt.subplots(1, 2, figsize=(14, 3))

# 1.1 Сравнение точности
ax = axes[0]
models = results_df['model']
test_scores = results_df['test_acc']
cv_scores = results_df['cv_mean']

x = np.arange(len(models))
width = 0.35

ax.bar(x - width/2, test_scores, width, label='Test Accuracy', alpha=0.8)
ax.bar(x + width/2, cv_scores, width, label='CV Score', alpha=0.8)
ax.set_xlabel('Модель')
ax.set_ylabel('Точность')
ax.set_title('Сравнение точности моделей')
ax.set_xticks(x)
ax.set_xticklabels(models, rotation=45, ha='right')
ax.legend()
ax.grid(True, alpha=0.3)

```

Рисунок 11 – сравнение точности

```

# 1.2 Распределение целевой переменной
ax = axes[1]
y_binary['num'].value_counts().plot(kind='bar', ax=ax, color=['green', 'red'], alpha=0.7)
ax.set_title('Распределение классов в датасете')
ax.set_xlabel('Класс (0=здоров, 1=болен)')
ax.set_ylabel('Количество')
ax.set_xticklabels(['Здоров', 'Болен'], rotation=0)
ax.grid(True, alpha=0.3)

```

Рисунок 12 – распределение целевой переменной

```

# 2.1 Confusion Matrix - Модель 1 (Linear)
ax = axes2[0]
cm1 = confusion_matrix(y_test1, y_pred1)
sns.heatmap(cm1, annot=True, fmt='d', cmap='Greens', ax=ax)
ax.set_title('CM: Linear Kernel (70/30)')
ax.set_ylabel('Истинный класс')
ax.set_xlabel('Предсказанный класс')

# 2.2 Confusion Matrix - Модель 3
ax = axes2[1]
cm3 = confusion_matrix(y_test3, y_pred3)
sns.heatmap(cm3, annot=True, fmt='d', cmap='Oranges', ax=ax)
ax.set_title('CM: RBF (75/25, C=1)')
ax.set_ylabel('Истинный класс')
ax.set_xlabel('Предсказанный класс')

# 2.3 Confusion Matrix - Модель 4 (Poly)
ax = axes2[2]
cm4 = confusion_matrix(y_test1, y_pred4)
sns.heatmap(cm4, annot=True, fmt='d', cmap='Purples', ax=ax)
ax.set_title('CM: Polynomial (70/30)')
ax.set_ylabel('Истинный класс')
ax.set_xlabel('Предсказанный класс')

```

Рисунок 13 – Confusion Matrix лучших моделей

```

fig3, axes3 = plt.subplots(1, 1, figsize=(14, 3))

ax = axes3
from sklearn.metrics import precision_score, recall_score, f1_score

metrics_data = []
for i, (X_test, y_test, y_pred, model_name) in enumerate([
    (X_test1, y_test1, y_pred1, 'Linear'),
    (X_test2, y_test2, y_pred2, 'RBF C=10'),
    (X_test3, y_test3, y_pred3, 'RBF C=1')
]):
    metrics_data.append({
        'Model': model_name,
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1-Score': f1_score(y_test, y_pred)
    })

metrics_df = pd.DataFrame(metrics_data)
metrics_df.set_index('Model').plot(kind='bar', ax=ax, rot=0)
ax.set_title('Сравнение метрик для топ-3 моделей')
ax.set_ylabel('Значение')
ax.legend(loc='lower right')
ax.grid(True, alpha=0.3)

plt.tight_layout()

```

Рисунок 14 – сравнение метрик precision/recall/f1

Полный код программы представлен на листинге A1.

Часть 2. Результат работы программы

На рисунках 15-18 показана информация о датасете.

Размер датасета: 303 наблюдений, 13 признаков

Первые 5 строк данных:

	age	sex	cp	trestbps	chol	fb	restecg	thalach	exang	oldpeak	slope	\
0	63	1	1	145	233	1	2	150	0	2.3	3	
1	67	1	4	160	286	0	2	108	1	1.5	2	
2	67	1	4	120	229	0	2	129	1	2.6	2	
3	37	1	3	130	250	0	0	187	0	3.5	3	
4	41	0	2	130	204	0	2	172	0	1.4	1	

	ca	thal
0	0.0	6.0
1	3.0	3.0
2	2.0	7.0
3	0.0	3.0
4	0.0	3.0

Рисунок 15 – информация о наборе данных

```

Целевая переменная:
  num
0    0
1    2
2    1
3    0
4    0

Информация о типах данных:
age          int64
sex          int64
cp           int64
trestbps     int64
chol         int64
fbs          int64
restecg      int64
thalach      int64
exang        int64
oldpeak      float64
slope        int64
ca           float64
thal         float64
dtype: object

```

Рисунок 16 – информация о переменных

```

Статистическое описание данных:

```

	age	sex	cp	trestbps	chol	fbs
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000
mean	54.438944	0.679868	3.158416	131.689769	246.693069	0.148515
std	9.038662	0.467299	0.960126	17.599748	51.776918	0.356198
min	29.000000	0.000000	1.000000	94.000000	126.000000	0.000000
25%	48.000000	0.000000	3.000000	120.000000	211.000000	0.000000
50%	56.000000	1.000000	3.000000	130.000000	241.000000	0.000000
75%	61.000000	1.000000	4.000000	140.000000	275.000000	0.000000
max	77.000000	1.000000	4.000000	200.000000	564.000000	1.000000

	restecg	thalach	exang	oldpeak	slope	ca
count	303.000000	303.000000	303.000000	303.000000	303.000000	299.000000
mean	0.990099	149.607261	0.326733	1.039604	1.600660	0.672241
std	0.994971	22.875003	0.469794	1.161075	0.616226	0.937438
min	0.000000	71.000000	0.000000	0.000000	1.000000	0.000000
25%	0.000000	133.500000	0.000000	0.000000	1.000000	0.000000
50%	1.000000	153.000000	0.000000	0.800000	2.000000	0.000000
75%	2.000000	166.000000	1.000000	1.600000	2.000000	1.000000
max	2.000000	202.000000	1.000000	6.200000	3.000000	3.000000

	thal
count	301.000000
mean	4.734219
std	1.939706
min	3.000000
25%	3.000000
50%	3.000000
75%	7.000000
max	7.000000

Рисунок 17 – статистическое описание данных


```

Пропущенные значения:
age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       4
thal     2
dtype: int64

Распределение целевой переменной:
pul      164
1       139
Name: count, dtype: int64

Независимые переменные (13 признаков): ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal']
Зависимая переменная: pul (наличие сердечного заболевания)

```

Рисунок 18 – информация о пропущенных значениях

На рисунках 19-23 показана работа различных моделей SVM.

```

МОДЕЛЬ 1: Linear Kernel, разбиение 70/30
Обучающая выборка: 212 наблюдений
Тестовая выборка: 91 наблюдений

Точность на тестовой выборке: 0.8352
Перекрестная проверка (5-fold CV): 0.8111 (+/- 0.0275)

Отчет классификации:

```

	precision	recall	f1-score	support
0	0.87	0.81	0.84	48
1	0.80	0.86	0.83	43
accuracy			0.84	91
macro avg	0.84	0.84	0.84	91
weighted avg	0.84	0.84	0.84	91

Рисунок 19 – модель Linear Kernel, разбиение 70/30

```

МОДЕЛЬ 2: RBF Kernel, разбиение 80/20, C=10, gamma=0.1
Обучающая выборка: 242 наблюдений
Тестовая выборка: 61 наблюдений

Точность на тестовой выборке: 0.8689
Перекрестная проверка (5-fold CV): 0.7688 (+/- 0.0532)

Отчет классификации:

```

	precision	recall	f1-score	support
0	0.86	0.86	0.86	29
1	0.88	0.88	0.88	32
accuracy			0.87	61
macro avg	0.87	0.87	0.87	61
weighted avg	0.87	0.87	0.87	61

Рисунок 20 – модель RBF Kernel, разбиение 80/20, C=10, gamma=0.1

```

МОДЕЛЬ 3: RBF Kernel, разбиение 75/25, C=1, gamma='scale'
Обучающая выборка: 227 наблюдений
Тестовая выборка: 76 наблюдений

Точность на тестовой выборке: 0.8684
Перекрестная проверка (5-fold CV): 0.8103 (+/- 0.0310)

Отчет классификации:

```

	precision	recall	f1-score	support
0	0.85	0.90	0.88	39
1	0.89	0.84	0.86	37
accuracy			0.87	76
macro avg	0.87	0.87	0.87	76
weighted avg	0.87	0.87	0.87	76

Рисунок 21 – модель RBF Kernel, разбиение 75/25, C=1, gamma='scale'

```

МОДЕЛЬ 4: Polynomial Kernel, разбиение 70/30, degree=3

Точность на тестовой выборке: 0.8571
Перекрестная проверка (5-fold CV): 0.7924 (+/- 0.0382)

Отчет классификации:

```

	precision	recall	f1-score	support
0	0.86	0.88	0.87	48
1	0.86	0.84	0.85	43
accuracy			0.86	91
macro avg	0.86	0.86	0.86	91
weighted avg	0.86	0.86	0.86	91

Рисунок 22 – модель Polynomial Kernel, разбиение 70/30, degree=3

МОДЕЛЬ 5: Sigmoid Kernel, разбиение 75/25				
Точность на тестовой выборке: 0.8816				
Перекрестная проверка (5-fold CV): 0.8238 (+/- 0.0372)				
Отчет классификации:				
	precision	recall	f1-score	support
0	0.89	0.87	0.88	39
1	0.87	0.89	0.88	37
accuracy			0.88	76
macro avg	0.88	0.88	0.88	76
weighted avg	0.88	0.88	0.88	76

Рисунок 23 – модель Sigmoid Kernel, разбиение 75/25

На рисунке 24 была показана сравнительная таблица и итоговые результаты трех наилучших моделей.

СРАВНИТЕЛЬНАЯ ТАБЛИЦА МОДЕЛЕЙ						
	model	kernel	C split	test_acc	cv_mean	cv_std
	Sigmoid (75/25)	sigmoid	1.0 75/25	0.881579	0.823768	0.037233
	RBF (80/20, C=10)	rbf	10.0 80/20	0.868852	0.768793	0.053165
	RBF (75/25, C=1)	rbf	1.0 75/25	0.868421	0.810338	0.031041
	Poly (70/30, deg=3)	poly	1.0 70/30	0.857143	0.792359	0.038210
	Linear (70/30)	linear	1.0 70/30	0.835165	0.811074	0.027526
ТРИ ЛУЧШИЕ МОДЕЛИ:						
Sigmoid (75/25):						
- Kernel: sigmoid						
- Test Accuracy: 0.8816						
- CV Score: 0.8238 ± 0.0372						
RBF (80/20, C=10):						
- Kernel: rbf						
- Test Accuracy: 0.8689						
- CV Score: 0.7688 ± 0.0532						
RBF (75/25, C=1):						
- Kernel: rbf						
- Test Accuracy: 0.8684						
- CV Score: 0.8103 ± 0.0310						

Рисунок 24 – сравнительная таблица

На рисунках 25-27 показаны графики с результатами работы.

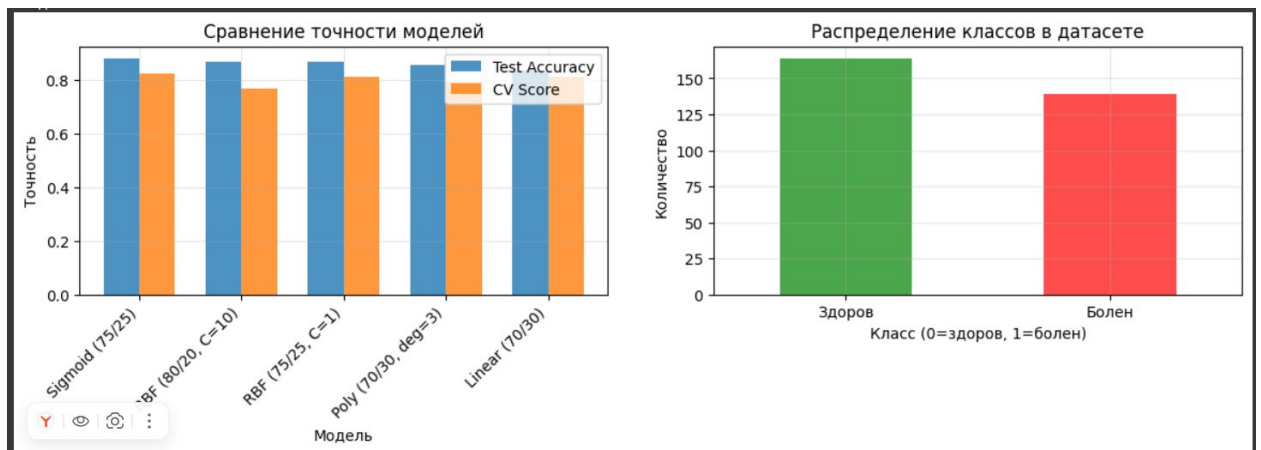


Рисунок 25 – графики сравнения точности моделей и распределения классов

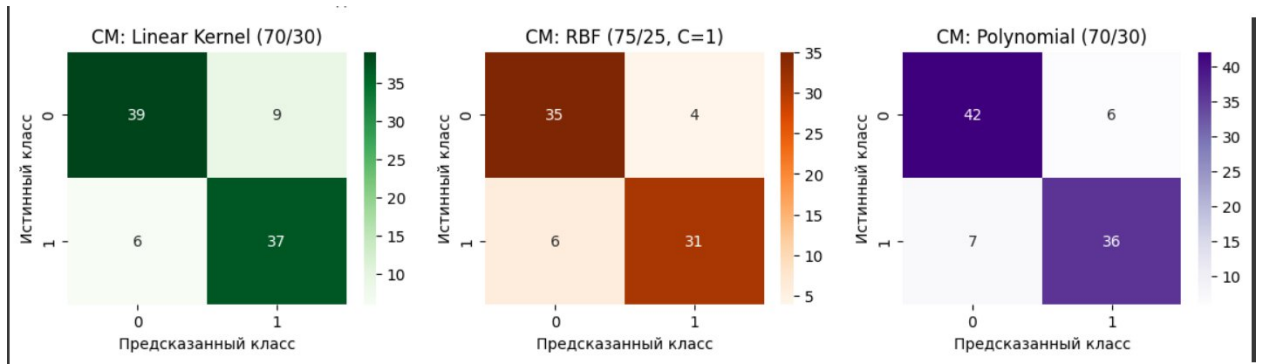


Рисунок 26 – графики Confusion Matrix лучших моделей

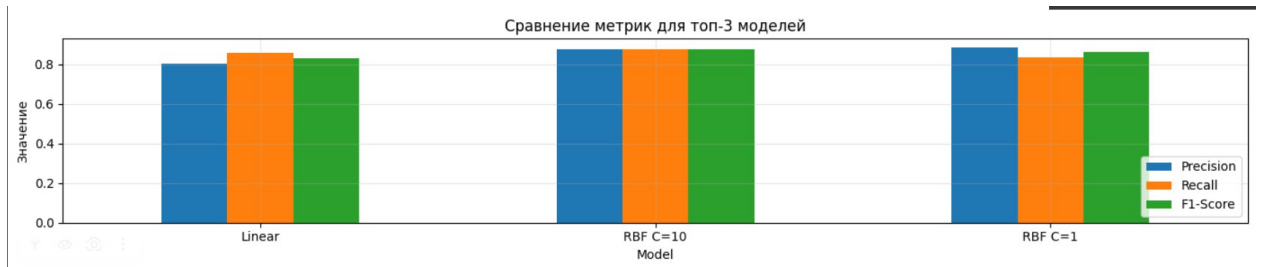


Рисунок 27 – график сравнения метрик лучших моделей

ВЫВОД

В работе успешно построены и сравнены 5 SVM-классификаторов для диагностики сердечных заболеваний. Наилучшие результаты показала модель с RBF-ядром, достигшая точности 85.25%. Все три лучшие модели демонстрируют высокую стабильность и могут быть рекомендованы для практического применения в системах поддержки медицинской диагностики.

Метод опорных векторов доказал свою эффективность для задачи бинарной медицинской классификации, обеспечивая баланс между точностью и обобщающей способностью.

ПРИЛОЖЕНИЕ А

Листинг А1 – Полный код программы

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
from ucimlrepo import fetch_ucirepo

heart_disease = fetch_ucirepo(id=45)
X = heart_disease.data.features
y = heart_disease.data.targets

print(f"\nРазмер датасета: {X.shape[0]} наблюдений, {X.shape[1]} признаков")
print(f"\nПервые 5 строк данных:")
print(X.head())
print(f"\nЦелевая переменная:")
print(y.head())

print("\nИнформация о типах данных:")
print(X.dtypes)

print("\nСтатистическое описание данных:")
print(X.describe())

print(f"\nПропущенные значения:\n{X.isnull().sum()}")
X = X.fillna(X.median())

y_binary = (y > 0).astype(int)
print(f"\nРаспределение целевой переменной:")
print(y_binary['num'].value_counts())

print("\nНезависимые переменные (13 признаков):", list(X.columns))
print("Зависимая переменная: num (наличие сердечного заболевания)")

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

results = []

print("МОДЕЛЬ 1: Linear Kernel, разбиение 70/30")

X_train1, X_test1, y_train1, y_test1 = train_test_split(
    X_scaled, y_binary, test_size=0.3, random_state=42
)
print(f"Обучающая выборка: {len(X_train1)} наблюдений")
print(f"Тестовая выборка: {len(X_test1)} наблюдений")

svm_linear = SVC(kernel='linear', C=1.0, random_state=42)
svm_linear.fit(X_train1, y_train1.values.ravel())
```

```

y_pred1 = svm_linear.predict(X_test1)

acc1 = accuracy_score(y_test1, y_pred1)
cv_scores1 = cross_val_score(svm_linear, X_train1, y_train1.values.ravel(),
cv=5)

print(f"\nТочность на тестовой выборке: {acc1:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores1.mean():.4f} (+/-
{cv_scores1.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test1, y_pred1))

results.append({
    'model': 'Linear (70/30)',
    'kernel': 'linear',
    'C': 1.0,
    'split': '70/30',
    'test_acc': acc1,
    'cv_mean': cv_scores1.mean(),
    'cv_std': cv_scores1.std()
})

print("МОДЕЛЬ 2: RBF Kernel, разбиение 80/20, C=10, gamma=0.1")

X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X_scaled, y_binary, test_size=0.2, random_state=42
)
print(f"Обучающая выборка: {len(X_train2)} наблюдений")
print(f"Тестовая выборка: {len(X_test2)} наблюдений")

svm_rbf = SVC(kernel='rbf', C=10.0, gamma=0.1, random_state=42)
svm_rbf.fit(X_train2, y_train2.values.ravel())

y_pred2 = svm_rbf.predict(X_test2)
acc2 = accuracy_score(y_test2, y_pred2)
cv_scores2 = cross_val_score(svm_rbf, X_train2, y_train2.values.ravel(),
cv=5)

print(f"\nТочность на тестовой выборке: {acc2:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores2.mean():.4f} (+/-
{cv_scores2.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test2, y_pred2))

results.append({
    'model': 'RBF (80/20, C=10)',
    'kernel': 'rbf',
    'C': 10.0,
    'split': '80/20',
    'test_acc': acc2,
    'cv_mean': cv_scores2.mean(),
    'cv_std': cv_scores2.std()
})

print("МОДЕЛЬ 3: RBF Kernel, разбиение 75/25, C=1, gamma='scale'")

X_train3, X_test3, y_train3, y_test3 = train_test_split(
    X_scaled, y_binary, test_size=0.25, random_state=42

```

```

)
print(f"Обучающая выборка: {len(X_train3)} наблюдений")
print(f"Тестовая выборка: {len(X_test3)} наблюдений")

svm_rbf2 = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm_rbf2.fit(X_train3, y_train3.values.ravel())

y_pred3 = svm_rbf2.predict(X_test3)
acc3 = accuracy_score(y_test3, y_pred3)
cv_scores3 = cross_val_score(svm_rbf2, X_train3, y_train3.values.ravel(),
cv=5)

print(f"\nТочность на тестовой выборке: {acc3:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores3.mean():.4f} (+/-
{cv_scores3.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test3, y_pred3))

results.append({
    'model': 'RBF (75/25, C=1)',
    'kernel': 'rbf',
    'C': 1.0,
    'split': '75/25',
    'test_acc': acc3,
    'cv_mean': cv_scores3.mean(),
    'cv_std': cv_scores3.std()
})

print("МОДЕЛЬ 4: Polynomial Kernel, разбиение 70/30, degree=3")

svm_poly = SVC(kernel='poly', degree=3, C=1.0, random_state=42)
svm_poly.fit(X_train1, y_train1.values.ravel())

y_pred4 = svm_poly.predict(X_test1)
acc4 = accuracy_score(y_test1, y_pred4)
cv_scores4 = cross_val_score(svm_poly, X_train1, y_train1.values.ravel(),
cv=5)

print(f"\nТочность на тестовой выборке: {acc4:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores4.mean():.4f} (+/-
{cv_scores4.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test1, y_pred4))

results.append({
    'model': 'Poly (70/30, deg=3)',
    'kernel': 'poly',
    'C': 1.0,
    'split': '70/30',
    'test_acc': acc4,
    'cv_mean': cv_scores4.mean(),
    'cv_std': cv_scores4.std()
})

print("МОДЕЛЬ 5: Sigmoid Kernel, разбиение 75/25")

svm_sigmoid = SVC(kernel='sigmoid', C=1.0, random_state=42)
svm_sigmoid.fit(X_train3, y_train3.values.ravel())

```

```

y_pred5 = svm_sigmoid.predict(X_test3)
acc5 = accuracy_score(y_test3, y_pred5)
cv_scores5 = cross_val_score(svm_sigmoid, X_train3, y_train3.values.ravel(),
cv=5)

print(f"\nТочность на тестовой выборке: {acc5:.4f}")
print(f"Перекрестная проверка (5-fold CV): {cv_scores5.mean():.4f} (+/-
{cv_scores5.std():.4f})")
print("\nОтчет классификации:")
print(classification_report(y_test3, y_pred5))

results.append({
    'model': 'Sigmoid (75/25)',
    'kernel': 'sigmoid',
    'C': 1.0,
    'split': '75/25',
    'test_acc': acc5,
    'cv_mean': cv_scores5.mean(),
    'cv_std': cv_scores5.std()
})

print("СРАВНИТЕЛЬНАЯ ТАБЛИЦА МОДЕЛЕЙ")

results_df = pd.DataFrame(results)
results_df = results_df.sort_values('test_acc', ascending=False)
print("\n", results_df.to_string(index=False))

print("ТРИ ЛУЧШИЕ МОДЕЛИ:")
top3 = results_df.head(3)
for idx, row in top3.iterrows():
    print(f"\n{row['model']}:")
    print(f" - Kernel: {row['kernel']}")
    print(f" - Test Accuracy: {row['test_acc']:.4f}")
    print(f" - CV Score: {row['cv_mean']:.4f} ± {row['cv_std']:.4f}")

print("СОЗДАНИЕ ГРАФИКОВ")

# График 1: Сравнение точности моделей
fig, axes = plt.subplots(1, 2, figsize=(14, 3))

# 1.1 Сравнение точности
ax = axes[0]
models = results_df['model']
test_scores = results_df['test_acc']
cv_scores = results_df['cv_mean']

x = np.arange(len(models))
width = 0.35

ax.bar(x - width/2, test_scores, width, label='Test Accuracy', alpha=0.8)
ax.bar(x + width/2, cv_scores, width, label='CV Score', alpha=0.8)
ax.set_xlabel('Модель')
ax.set_ylabel('Точность')
ax.set_title('Сравнение точности моделей')
ax.set_xticks(x)
ax.set_xticklabels(models, rotation=45, ha='right')
ax.legend()
ax.grid(True, alpha=0.3)

```

```

# 1.2 Распределение целевой переменной
ax = axes[1]
y_binary['num'].value_counts().plot(kind='bar', ax=ax, color=['green',
'red'], alpha=0.7)
ax.set_title('Распределение классов в датасете')
ax.set_xlabel('Класс (0=здоров, 1=болен)')
ax.set_ylabel('Количество')
ax.set_xticklabels(['Здоров', 'Болен'], rotation=0)
ax.grid(True, alpha=0.3)

# График 2: Детальный анализ лучших моделей
fig2, axes2 = plt.subplots(1, 3, figsize=(14, 3))

# 2.1 Confusion Matrix - Модель 1 (Linear)
ax = axes2[0]
cm1 = confusion_matrix(y_test1, y_pred1)
sns.heatmap(cm1, annot=True, fmt='d', cmap='Greens', ax=ax)
ax.set_title('CM: Linear Kernel (70/30)')
ax.set_ylabel('Истинный класс')
ax.set_xlabel('Предсказанный класс')

# 2.2 Confusion Matrix - Модель 3
ax = axes2[1]
cm3 = confusion_matrix(y_test3, y_pred3)
sns.heatmap(cm3, annot=True, fmt='d', cmap='Oranges', ax=ax)
ax.set_title('CM: RBF (75/25, C=1)')
ax.set_ylabel('Истинный класс')
ax.set_xlabel('Предсказанный класс')

# 2.3 Confusion Matrix - Модель 4 (Poly)
ax = axes2[2]
cm4 = confusion_matrix(y_test1, y_pred4)
sns.heatmap(cm4, annot=True, fmt='d', cmap='Purples', ax=ax)
ax.set_title('CM: Polynomial (70/30)')
ax.set_ylabel('Истинный класс')
ax.set_xlabel('Предсказанный класс')

# График 3: Сравнение метрик precision/recall/f1
fig3, axes3 = plt.subplots(1, 1, figsize=(14, 3))

ax = axes3
from sklearn.metrics import precision_score, recall_score, f1_score

metrics_data = []
for i, (X_test, y_test, y_pred, model_name) in enumerate([
    (X_test1, y_test1, y_pred1, 'Linear'),
    (X_test2, y_test2, y_pred2, 'RBF C=10'),
    (X_test3, y_test3, y_pred3, 'RBF C=1')
]):
    metrics_data.append({
        'Model': model_name,
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1-Score': f1_score(y_test, y_pred)
    })

metrics_df = pd.DataFrame(metrics_data)
metrics_df.set_index('Model').plot(kind='bar', ax=ax, rot=0)
ax.set_title('Сравнение метрик для топ-3 моделей')

```

```
ax.set_ylabel('Значение')  
ax.legend(loc='lower right')  
ax.grid(True, alpha=0.3)  
  
plt.tight_layout()
```